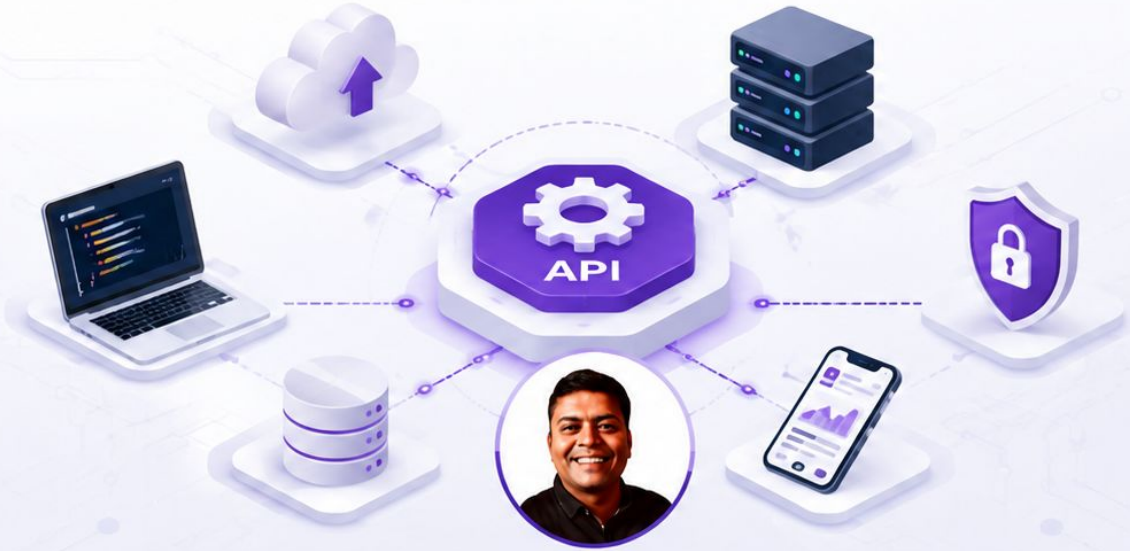
Designing APIs from Domain Models

Domain Modeling & Resource Design

API Design Mastery: Architecting Production-Ready APIs

Rahul Rajat Singh



Instructor: Rahul Rajat Singh

APIs Are Business Interfaces, Not Implementation Mirrors

- Stable APIs expose **business meaning**, not storage structures
- Database schemas and internal workflows **change faster** than contracts
- **Domain modeling** helps discover concepts worth making public
- Implementation-first APIs create **brittle coupling**



Implementation-First Approach

Leads to brittle coupling and higher change cost

Internal Data Model
Tied to storage structures

Implementation-Driven API
Exposes technical implementation

Consumer Coupling
High coupling and change impact

Business-First Approach

Leads to stable contracts and long-term value

Business Domain Model
Focused on business concepts

Business-Aligned API
Exposes business capabilities

Stable Contract
Low coupling and resilient to change

Instructor: Rahul Rajat Singh

BOUNDARIES MATTER: ONE BUSINESS DOES NOT MEAN ONE MODEL

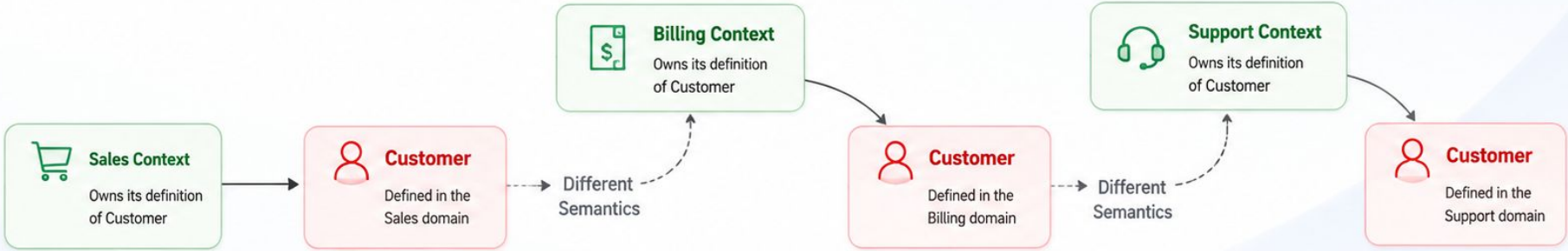


Same business terms often mean different things in different domains

"Customer" in billing is not always "customer" in support

Forced universal models create accidental coupling

APIs should respect conceptual ownership boundaries



Same name. Different meaning.

Respect boundaries. Design for clarity. Avoid accidental coupling.

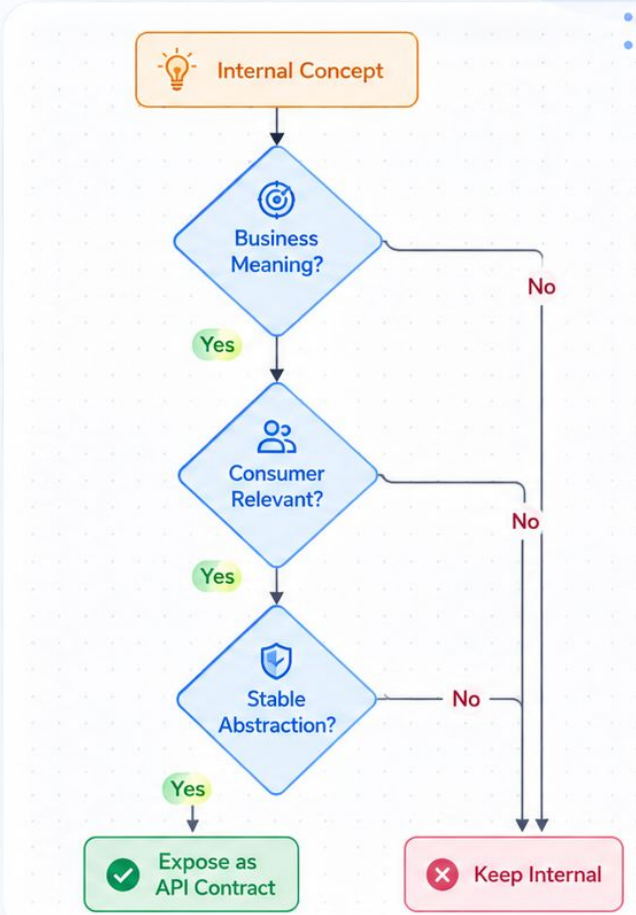
Discover API Resources from Business Capability

- What business capability are we exposing?
- Is this meaningful to consumers?
- Is this stable beyond current implementation?
- Are we exposing capability or internal mechanism?
- **Examples (Good):**
  - ✓ Order
  - ✓ Invoice
  - ✓ Shipment
  - ✓ Refund
- **Examples (Suspicious):**
  - ✗ WorkflowStep
  - ✗ EntityVersion
  - ✗ ProcessingNode









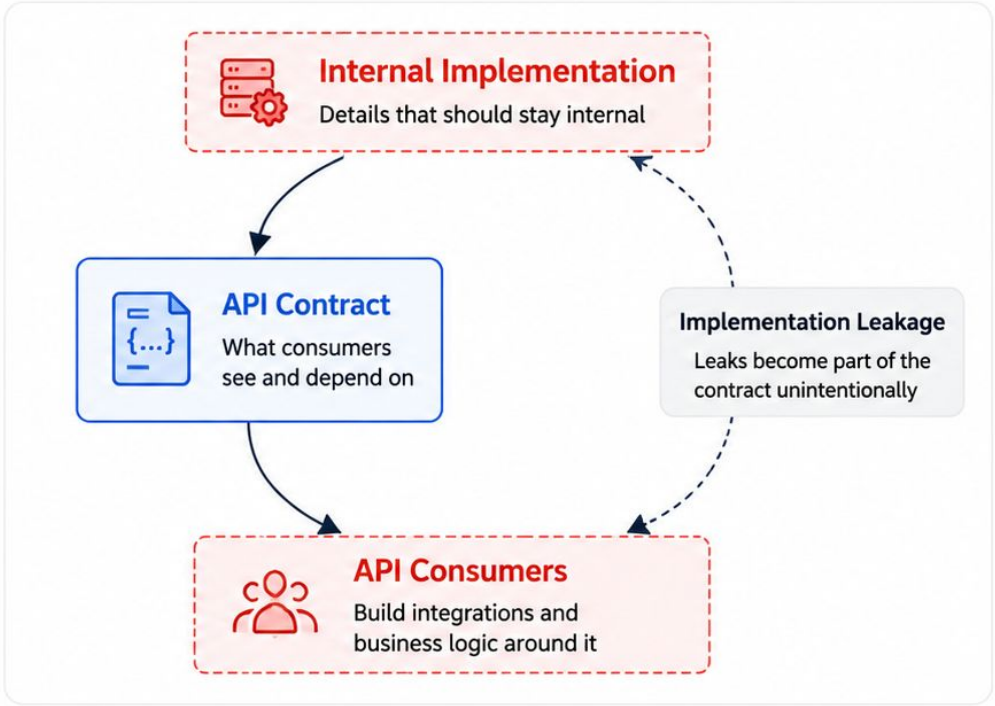
Technical Leakage Creates Long-Term API Debt

Leaky APIs expose:

- Persistence artifacts
- Workflow engine states
- Orchestration details
- Internal enums

Consequences:

- Migration paralysis
- Contract fragility
- Consumer misuse
- Versioning pain



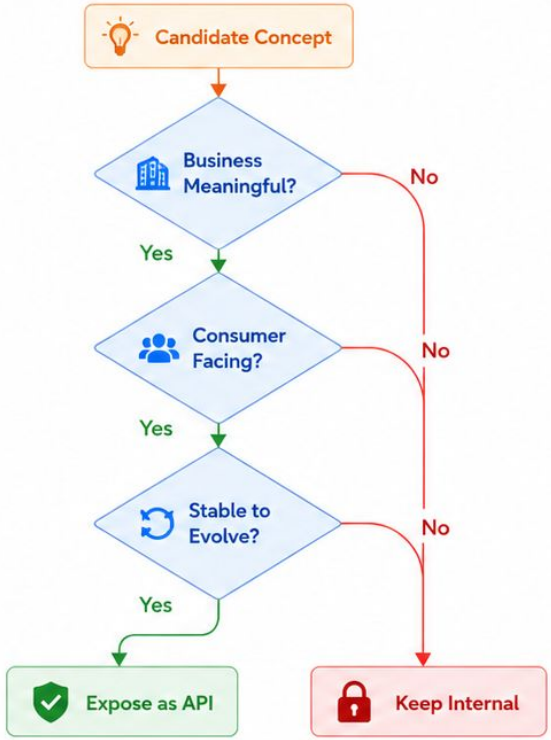
What leaks today, limits tomorrow.
Design APIs that **hide change**, not **expose it**.

Instructor: Rahul Rajat Singh

Architect Heuristic: Would This Concept Still Make Sense in 3 Years?

- Is this **business language** or **engineering language**?
- Who **owns** this concept?
- Would consumers **naturally understand** it?
- Does exposing this **increase coupling**?
- Are we **exporting complexity** instead of managing it?
- **Up Next:**
  - Resource Modeling Done Right

Aim for concepts that are **future-proof**, **clear to consumers**, and **easy to evolve**.



Resource Modeling Done Right

API Design Mastery:
Architecting Production-Ready APIs



Rahul Rajat Singh



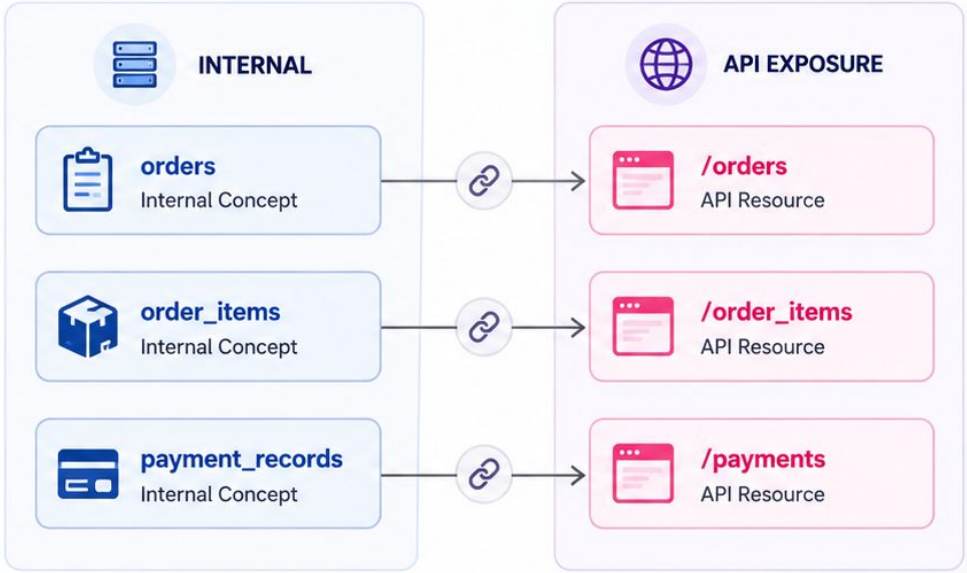
RESOURCE MODELING STARTS WITH BUSINESS LANGUAGE

Resources are business-facing abstractions
Model the language of the business, not the database.

APIs expose concepts, not persistence internals
Expose stable concepts with clear intent.

Contracts should survive implementation rewrites
Change the implementation, not the contract.

Database schemas are implementation details
They can (and will) evolve over time.



Model business concepts. Expose stable contracts. Hide implementation details.
That's how you design APIs that last.



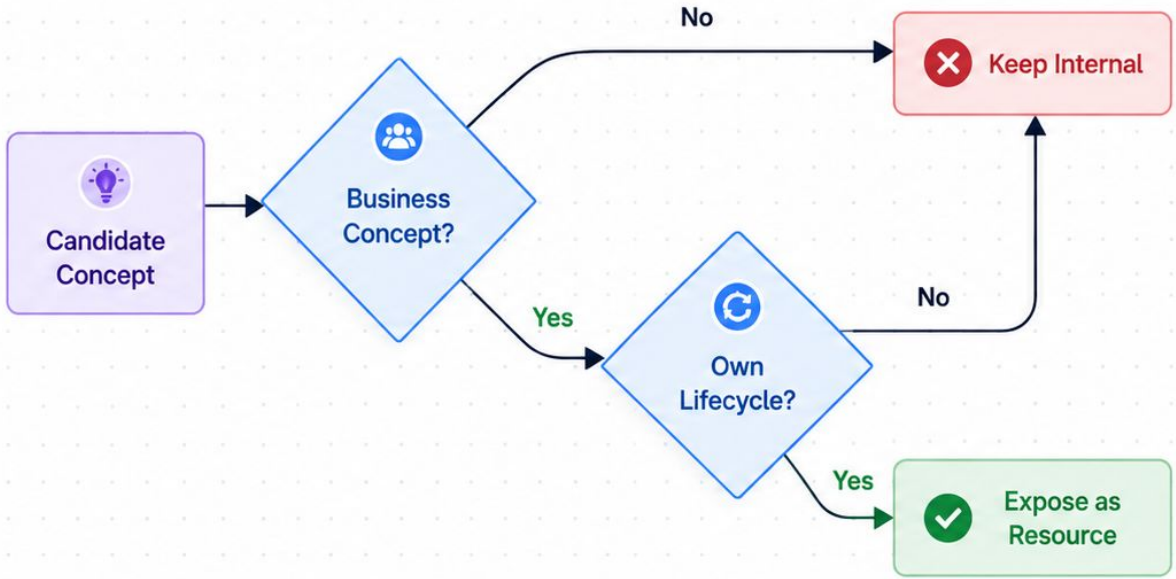
Not Every Domain Object Should Be a Resource

Expose only concepts with:

- Business meaning
- Stable identity
- Lifecycle ownership
- Consumer relevance

Bad examples:

- PermissionMapping
- UserRoleJoin
- TempSessionCache



Structure Should Reflect Ownership, Not Convenience

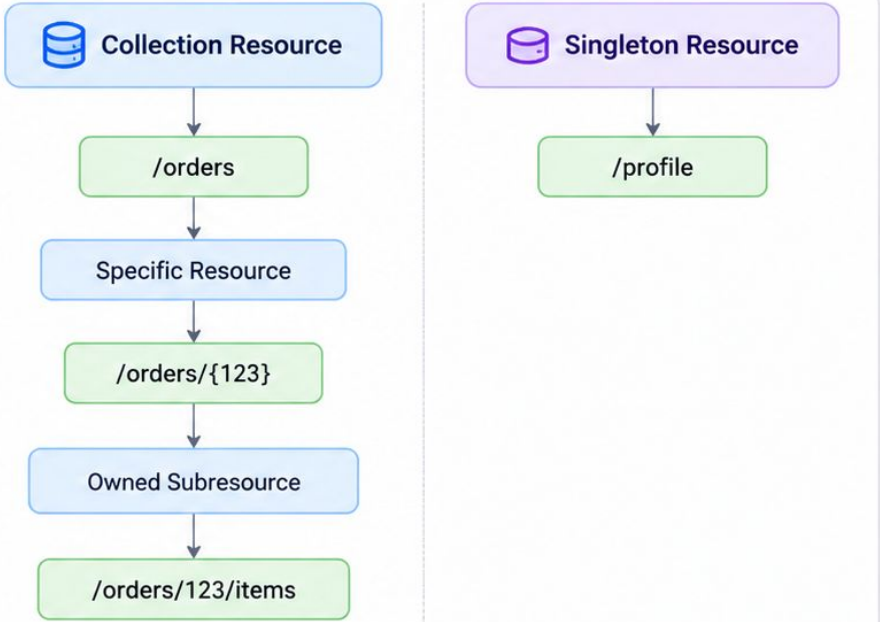
Use:

- Collections for entity sets
- Singletons for singular concepts
- Subresources for contained ownership

Examples:

- `/orders`
- `/profile`
- `/orders/{id}/items`

Avoid arbitrary nesting.



Design APIs with **clear ownership** and **logical structure.**

It makes your API more intuitive, scalable and maintainable.

Instructor: Rahul Rajat Singh

Relationships Do Not Automatically Mean Nested URLs

Association ≠ containment

- Customer has orders
- Order has payment
- Payment has refund

That does NOT mean:

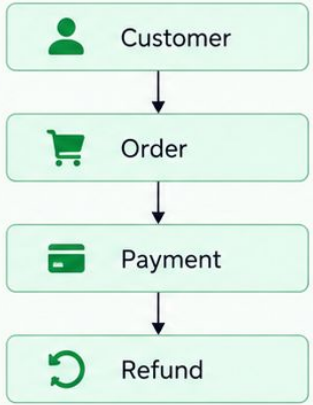
`/customers/42/orders/55/payments/9/refunds`

Deep nesting creates:

- path explosion
- access ambiguity
- authorization complexity

Model relationships with identifiers, not parent paths.
Design for clarity, flexibility, and change.

Relationship Without Over-Coupling



Over-Nested (False Containment)

`/customers/42/orders/55/payments/9/refunds`

Tight coupling, brittle, hard to evolve

Aggregate Boundaries

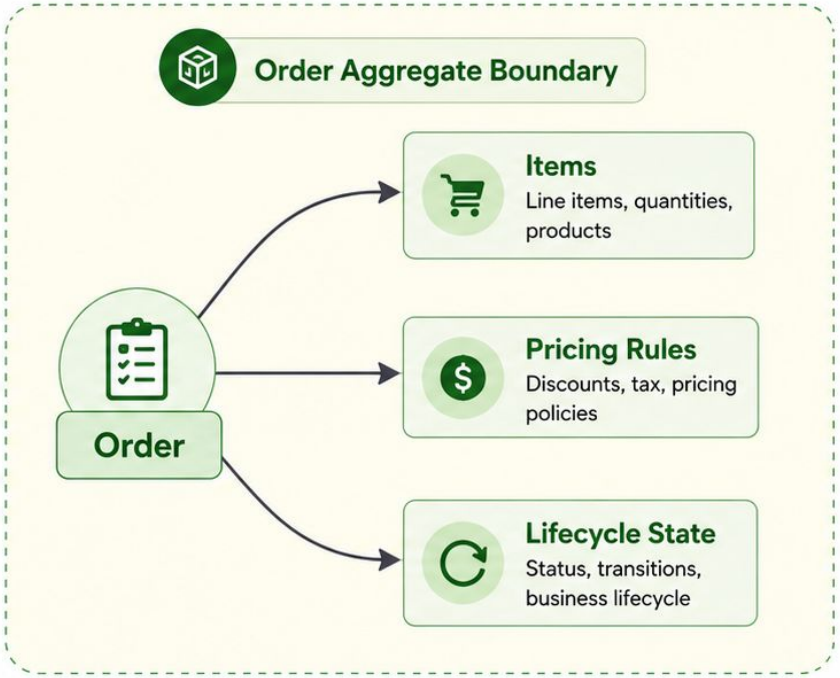
Define Resource Boundaries

Resources should align with **consistency boundaries.**

Example - **Order** includes:

- items
- totals
- status
- lifecycle rules

Avoid fragmenting one business transaction across many APIs.



Keep business transactions intact within an aggregate.
Consistency first. Boundaries second.



Canonical Resources Prevent Semantic Drift

One business entity should not appear as:

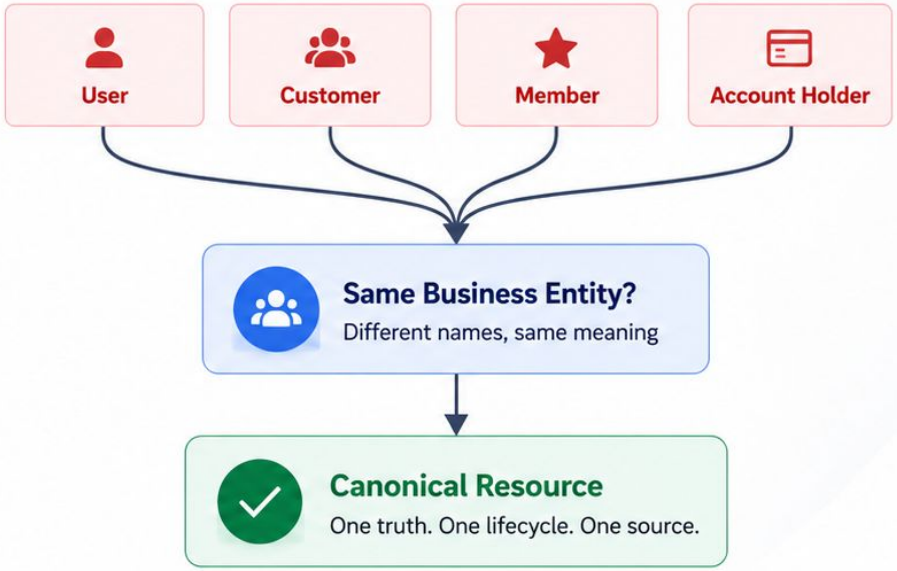
User | Customer | Member | Account Holder

Questions:

- Who owns truth?
- Which lifecycle is authoritative?

Outcome:

Define one canonical resource that captures the real-world meaning across the business.

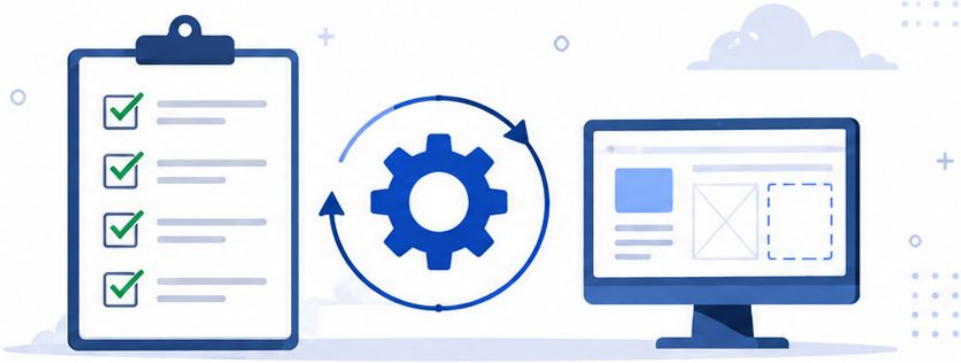


Different names create confusion. Canonical resources create clarity.
Define once. Use consistently. Govern centrally.



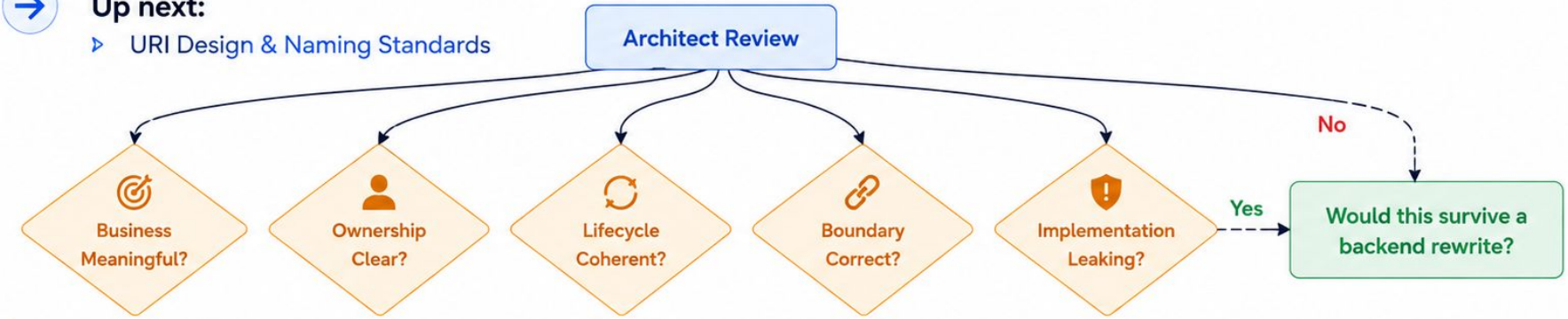
Architect Heuristic: Would This Resource Survive a Rewrite?

- business meaningful?
- ownership clear?
- lifecycle coherent?
- transaction boundary aligned?
- implementation leaked?



Up next:

- URI Design & Naming Standards



URI Design & Naming Standards

API Design Mastery: Architecting Production-Ready APIs

Rahul Rajat Singh

Clear & Consistent
Design for Understanding

Predictable & Scalable
Patterns that Grow

Production-Ready
Built for Reliability

Consumer-Centric
Simple, Intuitive, Valuable





URI Design Shapes API Usability

URIs are how consumers understand your **API model**.

Good URI design **reduces guesswork**.

Bad URI design increases **documentation dependency**.

Predictability improves **developer experience**.

customers/123/orders
Noun-based **resource**

→ **Resource-Oriented** →

Predictable API
Easy to understand, consistent and intuitive

getCustomerOrders
Verb-based **action**

→ **Action-Oriented** →

Endpoint Sprawl
Hard to scale, inconsistent and harder to maintain

Design URIs that reflect resources, not actions.
Clarity today. Flexibility tomorrow.



Resources, Not Actions

URIs identify business entities.

HTTP methods express intent.

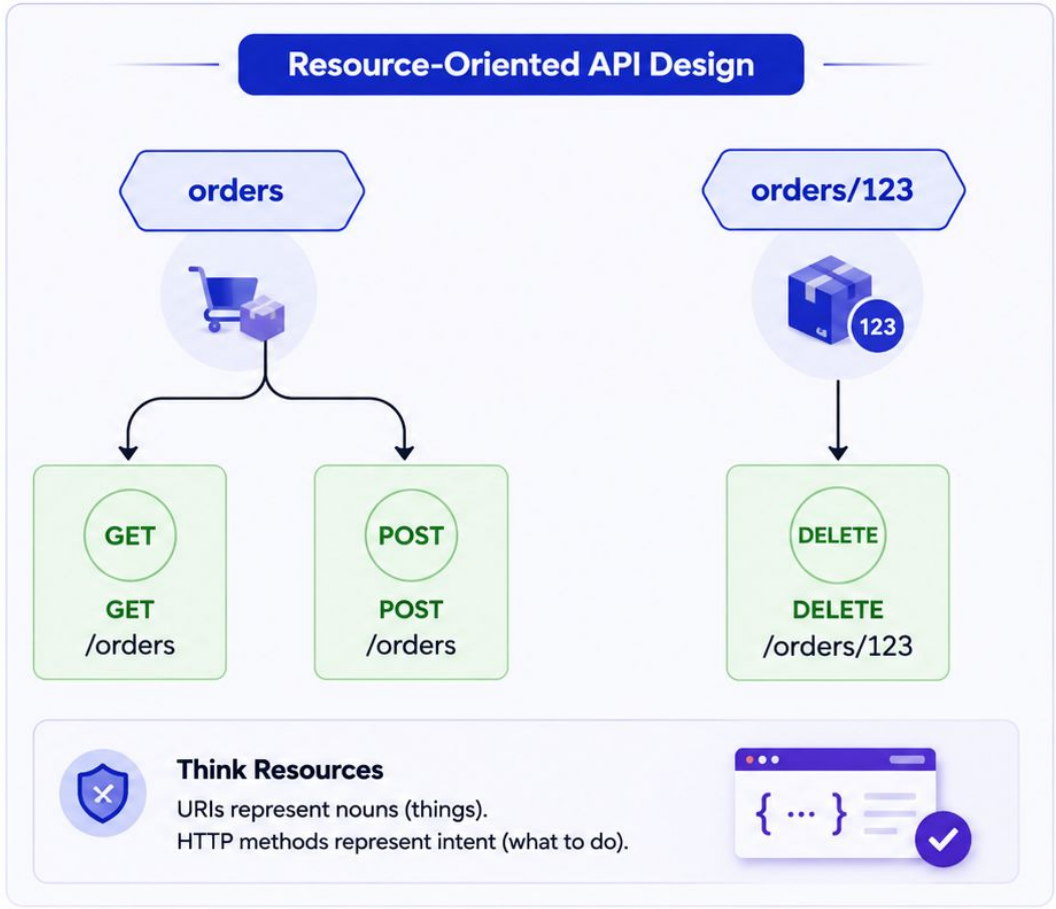
Examples:

- GET /orders
- POST /orders
- DELETE /orders/123

Avoid:

- /createOrder
- /cancelOrder
- /getInvoice

Design around nouns, not verbs.
Resources are stable. Actions change.



CONSISTENCY RULES MATTER MORE THAN NAMING PERFECTION



Pick standards and apply them everywhere:

- plural nouns
- lowercase paths
- hyphen-separated names
- no random abbreviations
- consistent terminology

Bad mix:

- /users
- /customer
- /acct
- /purchaseOrders

INCONSISTENT

- users
- customer
- acct
- purchaseOrders

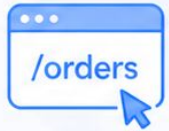
CONSISTENT

- customers
- purchase-orders
- user-profiles

Consistency > Perfection.
Choose standards once. **Apply them everywhere**. Your future self (and your teams) will thank you.

Instructor: Rahul Rajat Singh

URI Structure Should Reflect Business Meaning, Not System Design



- Good URIs reflect business concepts.

- Avoid leaking implementation:
  - /order-service
  - /mysql-orders
  - /payment-engine-v2

- Be careful with hierarchy:
  - Good: /customers/123/orders
  - Bad: /customers/123/orders/456/payments/7/refunds

Poor URI Design



Good URI Design

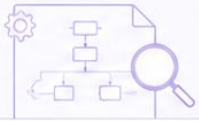


Architect Heuristic:
Can Consumers Predict the Next URI?

- Business meaning obvious?
- Verbs avoided?
- Naming consistent?
- Hierarchy justified?
- Implementation hidden?

Final heuristic: If consumers can **guess** the next endpoint correctly, your URI design is probably good.

Up Next:
Modeling Complex Business Operations



A PREDICTABLE URI JOURNEY

GET	/customers	List customers
GET	/customers/123	Get customer details
GET	/customers/123/orders	List customer's orders
GET	/customers/123/orders/456	Get order details
GET	/customers/123/orders/456/payments	List payments
GET	/customers/123/orders/456/refunds	List refunds

Predictable URIs = Intuitive APIs
Help consumers move forward with confidence.

Instructor: Rahul Rajat Singh

Modeling Complex Business Operations

API Design Mastery: Architecting Production-Ready APIs

Rahul Rajat Singh

- Model real-world business processes
- Design APIs that reflect business capabilities
- Build systems that scale with your business





> Design with clarity.
> Model with intent.
> Deliver with impact.

Instructor: Rahul Rajat Singh

WHEN CRUD STOPS MATCHING BUSINESS REALITY



Real business APIs expose **actions**, not just data records.

Refund, approve, onboard, retry are **business capabilities**.

Forcing everything into CRUD creates **awkward API design**.

CRUD-CENTRIC THINKING (THE PROBLEM)

PATCH /orders/123
status=approved

→ **State Mutation Thinking**
Update a field

→ **Leaked Internal State**
Implementation details exposed

→ **Weak Intent Semantics**
What is the real business action?

BUSINESS-ACTION THINKING (THE SOLUTION)

POST /orders/123/approve
Intent expressed clearly

→ **Business Action Thinking**
Model the capability, not the data

→ **Explicit Business Intent**
Clear purpose, easy to understand

→ **Clear Domain Contract**
Stable, meaningful, evolvable

Model business capabilities. Expose intent. Hide internal state.
Good APIs reflect how the business works — not how the database stores data.

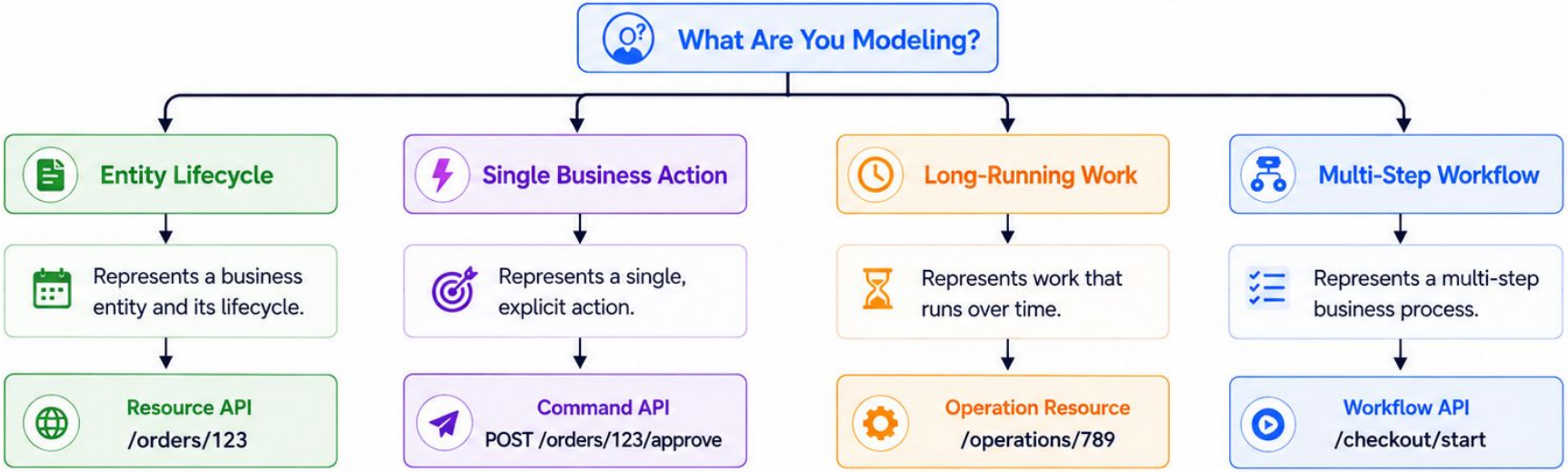


Instructor: Rahul Rajat Singh

Choose the Right Modeling Style



- **Resource** → business entity lifecycle
- **Command** → explicit action
- **Operation resource** → track long-running work
- **Workflow resource** → multi-step business process



Instructor: Rahul Rajat Singh

Operation Resources Make Async APIs Cleaner

Instead of:

- fire command
- hope it completes

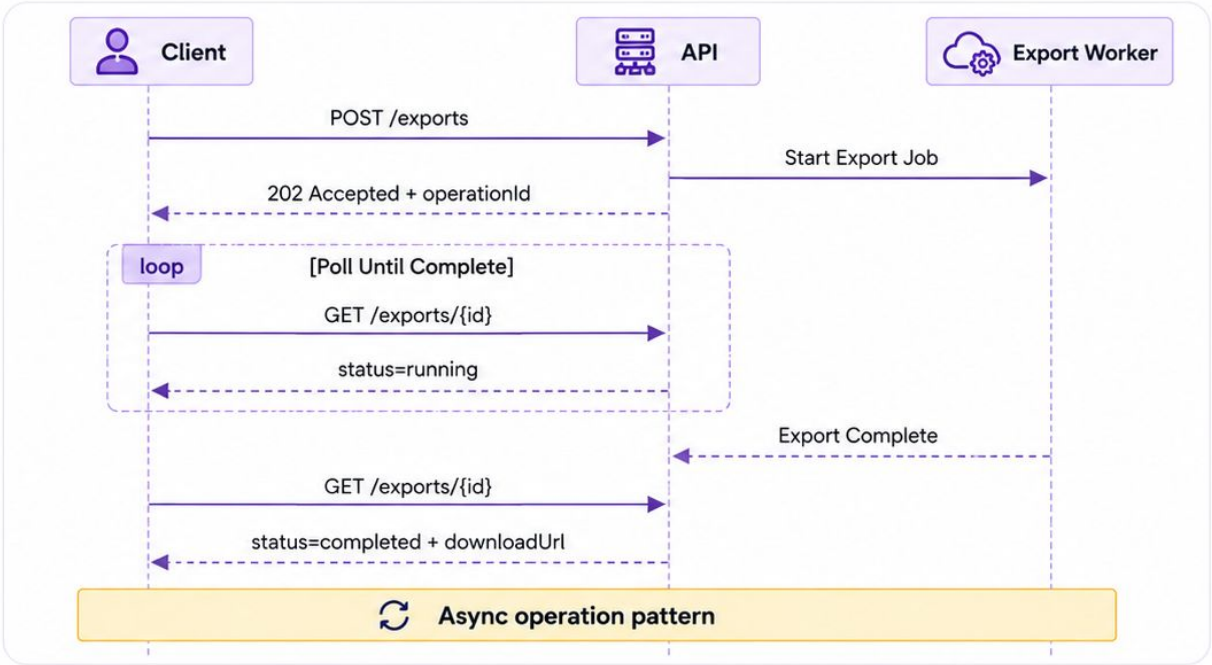
Prefer:

- create operation
- monitor status
- retrieve result

Examples:

exports

- report generation
- bulk imports
- provisioning



Use operation resources to make long-running tasks observable, reliable, and scalable.

Observable | Reliable | Scalable | Async by Design

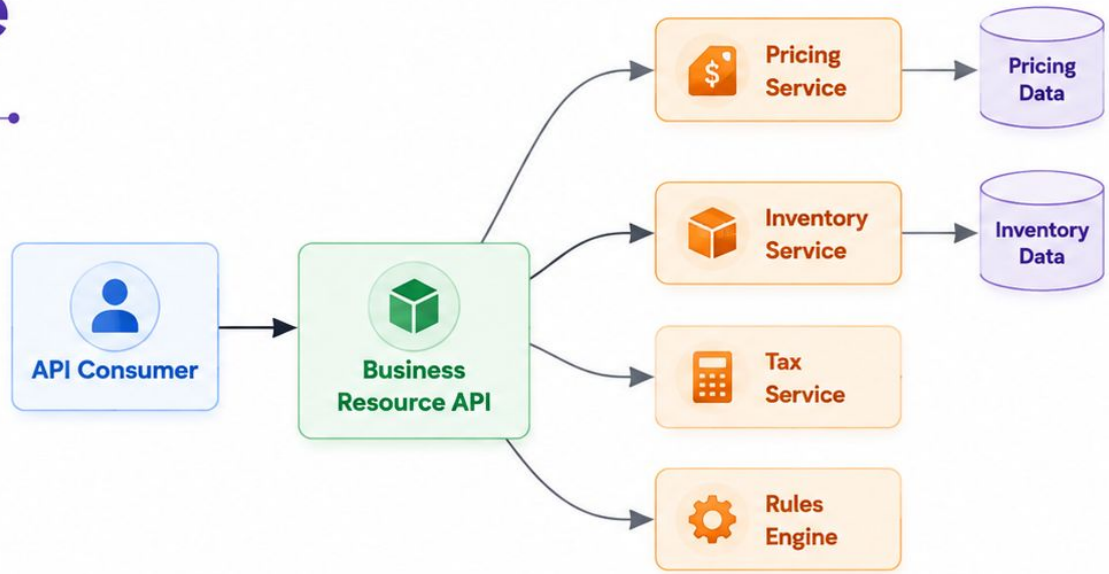
Not Every Resource Needs a Database Table

Valid API resources can be:

- computed
- temporary
- aggregated
- business-facing abstractions

Examples:

- quote
- invoice preview
- eligibility result
- checkout session



A resource is about **value to the consumer**, not about storage in a database.

Design resources that deliver real value.

Persistence is an implementation detail, not a requirement.

Instructor: Rahul Rajat Singh

Workflow APIs

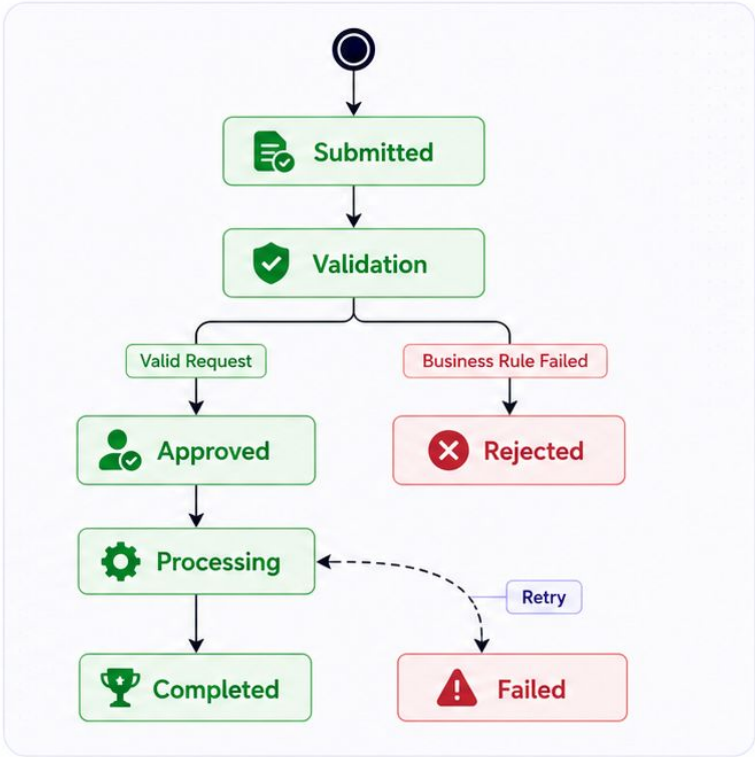
Represent Business Processes

Use workflow modeling when:

- Multiple states matter
- Approvals exist
- Transitions are constrained
- Lifecycle visibility matters

Examples:

- Claims
- Onboarding
- Loan Processing



Model the journey.

Expose the process, not just the data.

Clear Lifecycle | Visibility | Governance | Consistency

DISTRIBUTED REALITY BREAKS SIMPLISTIC API DESIGN

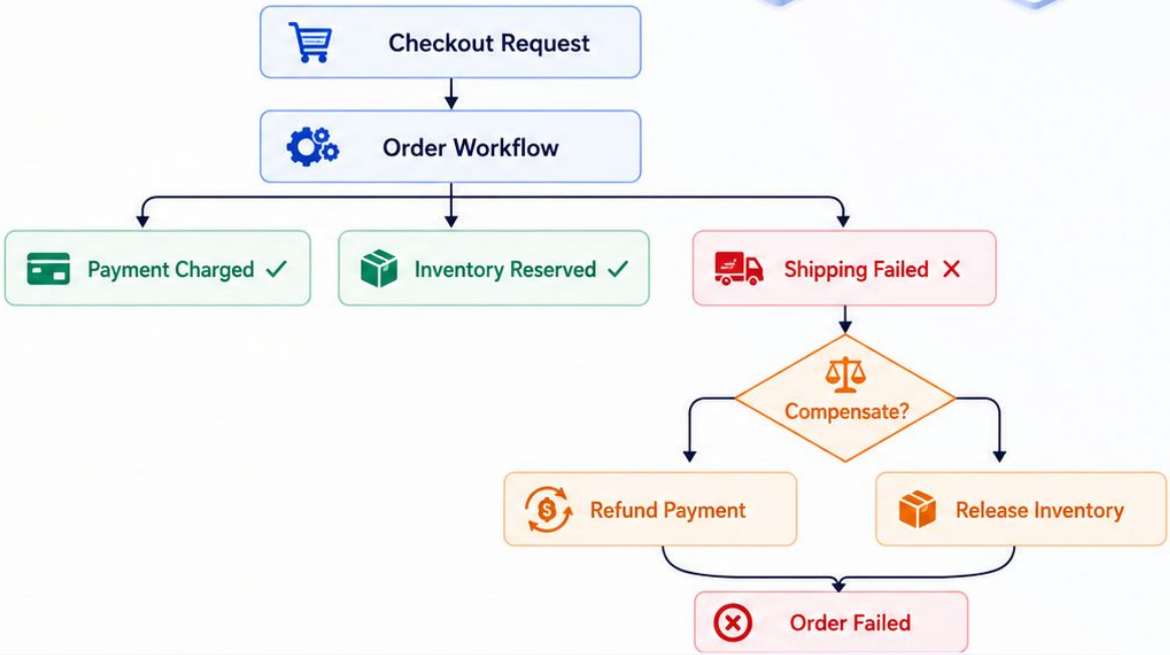


COMPLEX OPERATIONS CROSS SYSTEMS:

- payment
- inventory
- shipping
- notifications

FAILURES CREATE:

- partial completion
- duplicate actions
- inconsistent state



Distributed systems mean distributed failures.

Design APIs that handle uncertainty, retries, and compensation.

Instructor: Rahul Rajat Singh

Architect Heuristic: Model the Business Story Clearly

- Is this an **entity**, **action**, **operation**, or **workflow**?
- Does the API expose **business meaning**?
- Are **failure behaviors** understandable?
- Is **distributed complexity** hidden responsibly?
- Can consumers reason about **outcomes** safely?

Up Next:

> Common API Design Anti-Patterns



Instructor: Rahul Rajat Singh

Common
API Design Anti-Patterns

API Design Mastery: Architecting Production-Ready APIs

Rahul Rajat Singh

- Spot What Hurts
- Understand The Impact
- Avoid The Pitfalls
- Design Better APIs



Why Good Teams Still Build Bad APIs

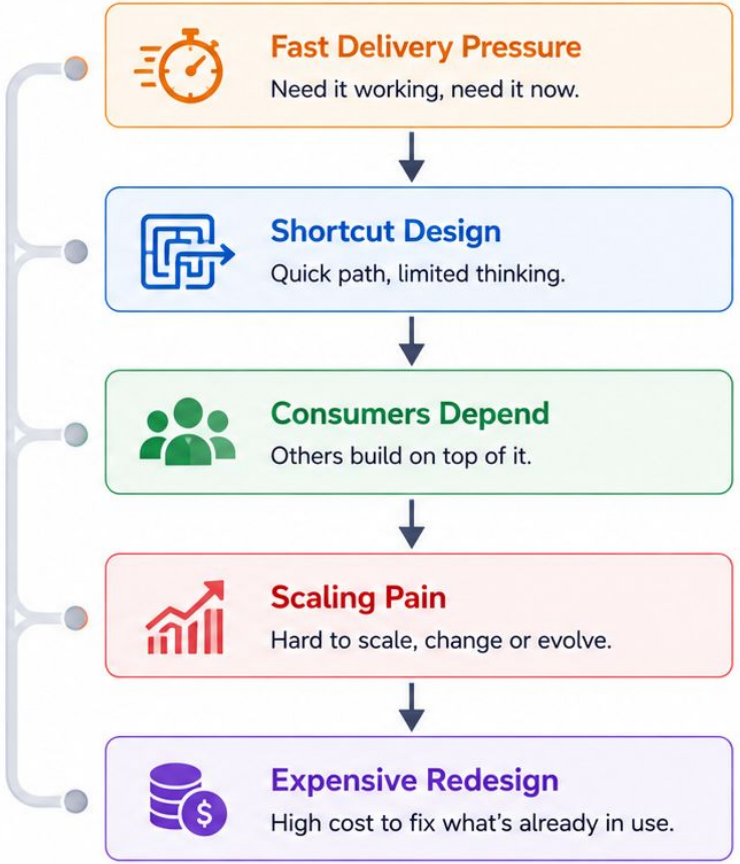
Bad APIs usually begin as **reasonable shortcuts.**

Implementation pressure shapes contracts.

Local optimization creates global pain.

Consumers inherit design mistakes.

APIs are contracts.
Shortcuts today become liabilities tomorrow.



Instructor: Rahul Rajat Singh

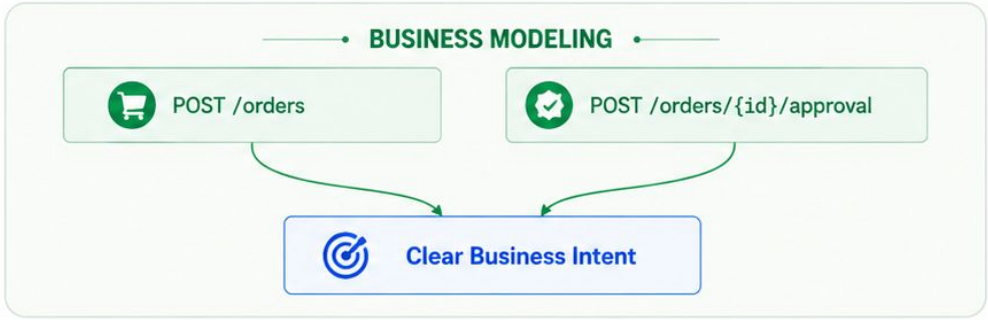
OPERATION-CENTRIC APIS: RPC, GOD ENDPOINTS, AND GENERIC COMMANDS

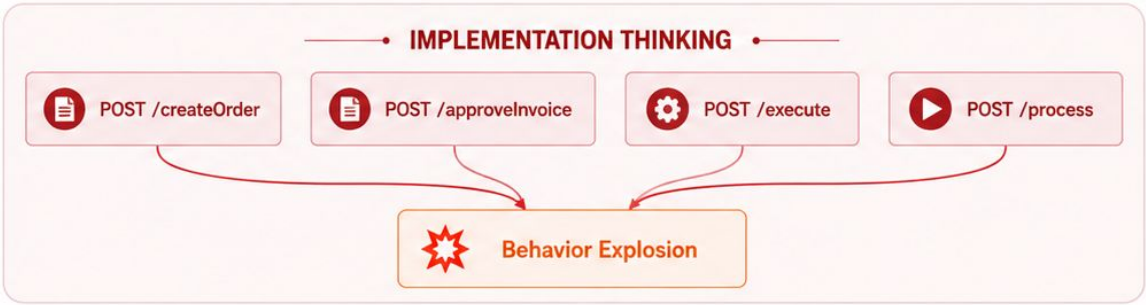
BAD EXAMPLES:

- POST `/createOrder`
- POST `/approveInvoice`
- POST `/execute`
- POST `/process`

PROBLEMS:

- business model unclear
- behavior hidden behind verbs
- contracts become ambiguous
- authorization becomes messy





Design around what the business means, not how the system works.

Intent first. Operations later.



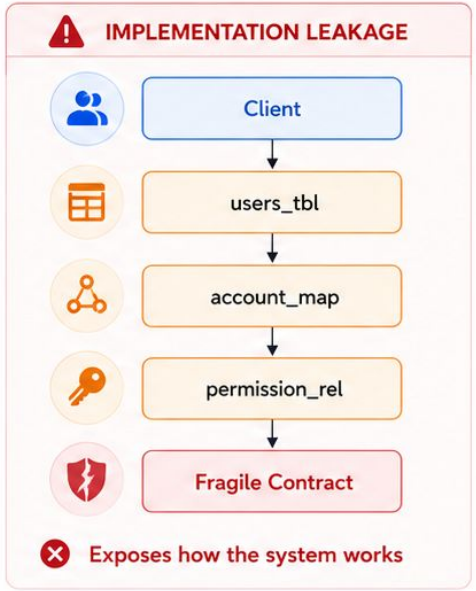
Instructor: Rahul Rajat Singh

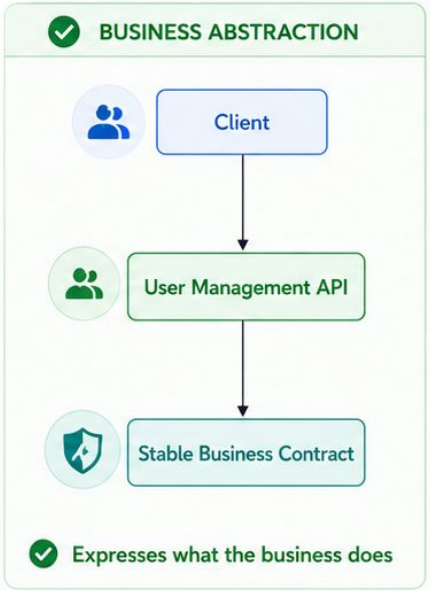
Implementation Leakage: APIs That Expose Internal Design

- database-shaped APIs
- join-table resource design
- deep nested URIs
- internal IDs everywhere
- transport concerns leaking into domain design

Bad example:

```
/customers/123/orders/456/items/789/discounts/22
```





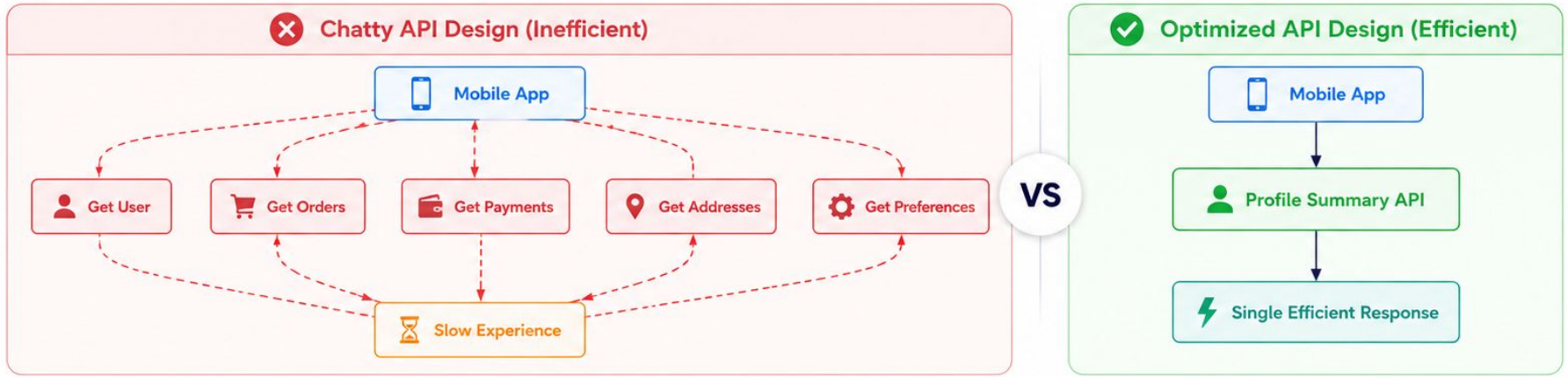
Great APIs **hide complexity** and expose **intent**, not implementation.



Instructor: Rahul Rajat Singh

Inefficient Interaction Design:
Chatty, Over-Fetching, Under-Fetching

- Too many round trips
- Oversized payloads
- Missing required data
- Consumers forced into orchestration

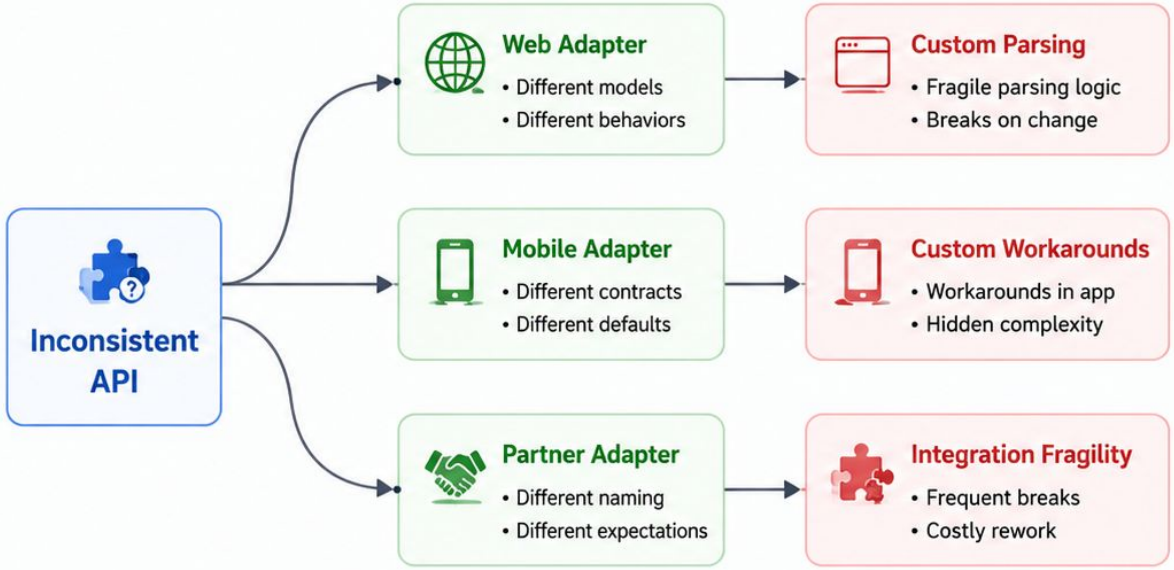


Design for clarity, efficiency, and consumer simplicity.
Fewer calls. Right data. Better experience.

Faster Performance | Smaller Payloads | Complete & Right Data | Consumer Simplicity

CONTRACT CHAOS: INCONSISTENCY AND TIGHT CONSUMER COUPLING

- inconsistent pagination
- inconsistent error shapes
- naming mismatches
- behavior differences between teams
- clients coding around quirks



Inconsistent contracts create chaos across every consumer.
Standardize. Align. Simplify.



Under-Modeled Business Workflows

Consumers orchestrating:

- order creation
- inventory reservation
- payment authorization
- shipment confirmation

Problems:

- distributed failure leakage
- retry ambiguity
- duplicate business actions
- inconsistent state

Final Heuristic:

- If consumers must understand your internal choreography, your API abstraction is wrong.

Next Section:

- HTTP Semantics & Contract Design

